CUN · Especialización en Inteligencia Artificial · Computación en la Nube

Informe del Notebook Reproducible — ACA N° 1

Proyecto: Sistema RAG cloud serverless en salud sobre Qdrant Cloud con embeddings abiertos.
Cuaderno de origen: **heart_disease_rag_qdrant.ipynb** · Colección: **clinical_heart_disease_rag** · Modelo: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2**

CORPORACIÓN UNIFICADA NACIONAL DE EDUCACIÓN SUPERIOR (CUN) Especialización en Inteligencia Artificial | Escuela de Ingeniería Asignatura: Computación en la Nube | Docente: Hamilton Rivera Flor --- PROYECTO ACA N° 1: SISTEMAS RAG EN CLÚSTER CLOUD SERVERLESS Ingesta, Vectorización y Almacenamiento Semántico en Qdrant Cloud — Pilar: Salud ■ | Componente | Selección | |---|---| | Dataset base (estructurado) | UCI Heart Disease (ID 45) — 303 pacientes, 14 variables clínicas | | Corpus complementario (no estructurado) | 3 guías clínicas en Markdown/PDF: MinSalud-OPS-OMS, ACC/AHA, ESC/OMS | | Base de datos vectorial | Qdrant Cloud (clúster serverless) — colección clinical_heart_disease_rag | | Modelo de embeddings | sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (384 dims) | | Fragmentación | RecursiveCharacterTextSplitter (LangChain) — chunk_size=512, chunk_overlap=64 | | Entorno | uv + Python 3.12 (entorno hermético y reproducible) | | Credenciales | Variables de entorno (.env) mediante python-dotenv — nunca en el código | --- ■ Objetivo General Diseñar e implementar una arquitectura cloud serverless de ing

0. Verificación de Dependencias y Entorno de Ejecución

Este cuaderno se ejecuta sobre un entorno gestionado con uv y Python 3.12, lo que garantiza reproducibilidad hermética: resolución determinista de dependencias y tiempos de instalación ultra rápidos. La celda siguiente prepara el sys.path para importar el paquete rag del repositorio (funciona igual en Jupyter Lab local, VS Code o Google Colab con el repo clonado).

```
# 0. Preparación del entorno: locate el repositorio e importar el paquete `rag`
import os
import sys
import logging
from pathlib import Path

# --- Localizar la raíz del repositorio (contiene pyproject.toml) -----------------------
def find_repo_root(start: Path) -> Path:
    for candidate in [start, *start.parents]:
        if (candidate / "pyproject.toml").exists() and (candidate / "src" / "rag").is_dir():
            return candidate
    return start

REPO_ROOT = find_repo_root(Path.cwd())
SRC_PATH = REPO_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))

# En Google Colab el repositorio se puede clonar con:
#   !git clone https://github.com/jaimeacabreraa/computacion_en_la_nube.git
#   %cd computacion_en_la_nube

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate
from IPython.display import Markdown, display

from rag.config import settings
from rag.dataset_loader import load_heart_disease_dataset, generate_patient_corpus
from rag.document_processor import load_unstructured_documents, chunk_unstructured_documents
from rag.vector_store import (
    get_qdrant_client,
    create_or_recreate_collection,
    index_corpus_in_batches,
    semantic_search,
)
from rag.evaluation import BENCHMARK_QUERIES, run_retrieval_benchmark, summarize_results

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
sns.set_theme(style="whitegrid", palette="muted")

print("■ Stack tecnológico cargado.")
print(f"   Repositorio        : {REPO_ROOT}")
print(f"   Modo de ejecución  : {'CLOUD (Qdrant Cloud)' if settings.is_cloud_mode else 'LOCAL (in-memo
ry)'}")
print(f"   Colección          : {settings.collection_name}")
print(f"   Modelo embeddings  : {settings.embedding_model_name}")
print(f"   Chunking           : chunk_size={settings.chunk_size}, chunk_overlap={settings.chunk_overlap
}")
```

```
■ Stack tecnológico cargado.
Repositorio : /home/jaimeacabreraa/Documentos/computacion_en_la_nube
Modo de ejecución : CLOUD (Qdrant Cloud)
Colección : clinical_heart_disease_rag
Modelo embeddings : sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Chunking : chunk_size=512, chunk_overlap=64
```

SECCIÓN 1: Aprovisionamiento Cloud y Conectividad Segura
Criterio de rúbrica 1 (20%): Aprovisionamiento Cloud y Conectividad

En cumplimiento estricto de las directrices de seguridad cloud del ACA 1:
• Las credenciales (QDRANT_URL y QDRANT_API_KEY) se leen exclusivamente desde variables de entorno (.env) mediante python-dotenv. No hay tokens, ni URLs privadas, ni secretos hardcodeados en el cuaderno ni en el repositorio (el .env está en .gitignore).
• Se aprovisiona la colección vectorial en Qdrant Cloud con:
• Dimensionalidad 384, correspondiente al espacio latente de paraphrase-multilingual-MiniLM-L12-v2.
• Métrica de distancia Cosine (Distance.COSINE), coherente con vectores normalizados en L2.

• Índices de payload nativos sobre doc_type, sex, disease_present, risk_level, patient_id y age: permiten filtrado estructurado combinable con la búsqueda vectorial sin penalizar la latencia.

```
# 1.1 Conexión segura al clúster de Qdrant Cloud (sin exponer la API key)
client = get_qdrant_client()

print("■ Endpoint Qdrant :", settings.qdrant_url.replace(settings.qdrant_url.split(".")[0], "https://<
cluster-id>")
      if settings.is_cloud_mode else "In-Memory (local)")
print("■ API Key        :", "[REDACTED — leída desde .env]" if settings.qdrant_api_key else "No config
urada (modo local)")
print("■ Colección     :", settings.collection_name)

cluster_status = client.get_collections()
print("■ Colecciones existentes en el clúster:", [c.name for c in cluster_status.collections])
```

```
2026-10-22 00:05:26,527 [INFO] Conectando a Qdrant Cloud en
https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-1-0.aws.cloud.qdrant.io...
■ Endpoint Qdrant : https://<cluster-id>.ca-central-1-0.aws.cloud.qdrant.io
■ API Key : [REDACTED — leída desde .env]
■ Colección : clinical_heart_disease_rag
2026-10-22 00:05:27,225 [INFO] HTTP Request: GET
https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-1-0.aws.cloud.qdrant.io:6333 "HTTP/1.1 200 OK"
2026-10-22 00:05:27,227 [INFO] HTTP Request: GET
https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-1-0.aws.cloud.qdrant.io:6333/collections
"HTTP/1.1 200 OK"
■ Colecciones existentes en el clúster: ['clinical_heart_disease_rag']
```

```
# 1.2 Aprovisionamiento de la colección vectorial (métrica coseno + índices de payload)
create_or_recreate_collection(client)

info = client.get_collection(settings.collection_name)
print(f"■ Colección '{settings.collection_name}' aprovisionada | "
      f"dim={info.config.params.vectors.size} | métrica={info.config.params.vectors.distance}")
print(f"   Vectores actuales en el clúster: {info.points_count}")
```

```
2026-10-22 00:05:27,351 [INFO] HTTP Request: GET
https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-1-0.aws.cloud.qdrant.io:6333/collections
"HTTP/1.1 200 OK"
2026-10-22 00:05:27,353 [INFO] Recreando colección existente: clinical_heart_disease_rag
2026-10-22 00:05:27,532 [INFO] HTTP Request: DELETE https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-centr
al-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag "HTTP/1.1 200 OK"
2026-10-22 00:05:27,797 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag "HTTP/1.1 200 OK"
2026-10-22 00:05:27,799 [INFO] Colección 'clinical_heart_disease_rag' creada exitosamente (size=384,
distance=COSINE).
2026-10-22 00:05:28,012 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:28,015 [INFO] Índice de payload creado para 'doc_type' (keyword).
2026-10-22 00:05:28,248 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:28,251 [INFO] Índice de payload creado para 'sex' (keyword).
2026-10-22 00:05:28,453 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:28,455 [INFO] Índice de payload creado para 'disease_present' (bool).
2026-10-22 00:05:28,657 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:28,659 [INFO] Índice de payload creado para 'risk_level' (keyword).
2026-10-22 00:05:28,887 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:28,889 [INFO] Índice de payload creado para 'patient_id' (integer).
2026-10-22 00:05:29,118 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/index?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:29,120 [INFO] Índice de payload creado para 'age' (integer).
2026-10-22 00:05:29,272 [INFO] HTTP Request: GET https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag "HTTP/1.1 200 OK"
■ Colección 'clinical_heart_disease_rag' aprovisionada | dim=384 | métrica=Cosine
Vectores actuales en el clúster: 0
```

SECCIÓN 2: Calidad de Datos, Ingesta Multimodal y Estrategia de Chunking
Criterio de rúbrica 2 (20%): Calidad de Datos y Chunking

Esta sección cubre los tres elementos que exige la rúbrica:

1. Ingesta estructurada: descarga del dataset UCI Heart Disease (ID 45) con la librería oficial ucimlrepo (reproducible, sin CSV manual).
2. Narrativización y enriquecimiento de metadatos: cada fila tabular (códigos numéricos como cp=1 o thal=7) se convierte en una viñeta clínica en lenguaje natural y se acompaña de metadatos interpretables (age, sex, risk_level, disease_present, major_vessels...).
Sin este paso, un RAG no podría responder consultas en lenguaje natural sobre datos tabulares.
3. Corpus documental no estructurado (obligatorio): 3 guías clínicas de autoridad, en Markdown y PDF, que son fragmentadas, vectorizadas y consultadas.

Justificación técnica del chunk_size = 512 y chunk_overlap = 64

Parámetro	Valor	Justificación
chunk_size	512 caracteres	Alineado con la ventana de 256 *wordpiece tokens* del encoder Sentence-BERT: el fragmento completo cabe en el contexto, sin truncamiento semántico ni pérdida de la oración.
chunk_overlap	64 caracteres (~12,5 %)	Evita que una afirmación que cae justo en la frontera de corte se parta en dos fragmentos incomprensibles.
separators	["\n\n", "\n", ". ", " ", ""]	Jerarquía de cortes: primero párrafos, luego líneas, luego oraciones, y solo al final palabras. Así se respeta la estructura semántica del texto.

```
# 2.1 Descarga del dataset base estructurado (UCI Heart Disease, ID 45)
df_raw, dataset_metadata = load_heart_disease_dataset()

print(f"■ Dataset cargado: {df_raw.shape[0]} pacientes x {df_raw.shape[1]} variables")
print(f"■ Fuente: {dataset_metadata.get('name', 'UCI Heart Disease')}")
print("\n■ Distribución del diagnóstico (0 = sin cardiopatía, 1-4 = cardiopatía isquémica):")
display(df_raw["target"].value_counts().sort_index().to_frame("Pacientes"))

print("\n■ Primeras 3 filas del dataset crudo:")
display(df_raw.head(3))
```

```
■ Dataset cargado: 303 pacientes x 15 variables
■ Fuente: Heart Disease

■ Distribución del diagnóstico (0 = sin cardiopatía, 1-4 = cardiopatía isquémica):
■ Primeras 3 filas del dataset crudo:
```

```
# 2.2 Transformación tabular -> viñeta clínica con metadatos enriquecidos
patient_corpus = generate_patient_corpus(df_raw)

ejemplo = patient_corpus[0]
print("=" * 90)
print("EJEMPLO DE VIÑETA CLÍNICA GENERADA (Paciente #001)")
print("=" * 90)
print(ejemplo["text"])
print("-" * 90)
print("METADATOS ASOCIADOS:")
for clave, valor in ejemplo["metadata"].items():
    print(f"   {clave:<24}: {valor}")
```

```
================================================================================
EJEMPLO DE VIÑETA CLÍNICA GENERADA (Paciente #001)
================================================================================
HISTORIA CLÍNICA CARDIOLÓGICA - PACIENTE #001
Datos Demográficos: Paciente de 63 años de edad, sexo Masculino.
Motivo de Consulta y Dolor Torácico: Angina típica (Tipo 1, dolor retroesternal opresivo).
Constantes y Perfil Metabólico: Presión arterial sistólica en reposo de 145 mmHg; colesterol sérico total
de 233 mg/dL; Glucemia elevada >120 mg/dl (diabetes/intolerancia).
Electrocardiograma Basal: Hipertrofia ventricular izquierda (HVI).
Evaluación Funcional y Prueba de Esfuerzo (Ergometría): Frecuencia cardíaca pico alcanzada de 150 lpm;
Angina inducida por esfuerzo: Ausente en el esfuerzo; Depresión del segmento ST de 2.3 mm con morfología
de Pendiente descendente (Downsloping, isquemia severa).
Estudios Invasivos y Perfusión: Angiografía fluoroscópica con calcificación/obstrucción en 0 vasos
mayores; Estudio de perfusión SPECT: Defecto de perfusión fijo (necrosis / infarto antiguo).
Diagnóstico Clínico de Cardiopatía Isquémica: AUSENCIA (Estadio de severidad angiográfica: 0/4). Nivel de
Riesgo Global: Moderado.
--------------------------------------------------------------------------------
METADATOS ASOCIADOS:
doc_type : patient_record
patient_id : 1
age : 63
age_group : 50-64
sex : Masculino
chest_pain_type : 1
chest_pain_label : Angina típica
resting_blood_pressure : 145
cholesterol : 233
fasting_blood_sugar : 1
max_heart_rate : 150
exercise_angina : 0
st_depression : 2.3
st_slope : 3
major_vessels : 0
thal_status : 6
target_disease : 0
disease_present : False
risk_level : Moderado
```

```
# 2.3 Carga y fragmentación del corpus documental complementario (no estructurado)
raw_docs = load_unstructured_documents(str(settings.documents_dir))
chunked_docs = chunk_unstructured_documents(raw_docs)

print(f"■ Documentos complementarios cargados: {len(raw_docs)}")
for doc in raw_docs:
    n_chunks = sum(1 for c in chunked_docs if c["metadata"]["filename"] == doc["filename"])
    print(f"   • {doc['filename']}")
    print(f"     entidad={doc['metadata']['entity']} | doc_type={doc['metadata']['doc_type']} "
          f"| caracteres={len(doc['content']):,} | fragmentos={n_chunks}")

print(f"\n■ Fragmentos documentales generados: {len(chunked_docs)}")
print(f"■ Longitud media de fragmento: {np.mean([c['metadata']['chunk_char_length'] for c in chunked_d
ocs]):.1f} caracteres")
print(f"■ Corpus total a indexar: {len(chunked_docs)} fragmentos documentales + {len(patient_corpus)}
viñetas clínicas = {len(chunked_docs) + len(patient_corpus)} vectores")
```

```
■ Documentos complementarios cargados: 3
• DOC_01_Guia_Clinica_Enfermedad_Coronaria_MinSalud_OMS.md
entidad=MinSalud / OPS / OMS | doc_type=clinical_guideline | caracteres=4,846 | fragmentos=15
• DOC_02_Protocolo_Pruebas_Esfuerzo_Ergometria_ACC_AHA.md
entidad=ACC / AHA | doc_type=diagnostic_protocol | caracteres=5,379 | fragmentos=18
• DOC_03_Guia_Manejo_Farmacologico_Prevencion_Secundaria.md
entidad=ESC / OMS | doc_type=pharmacology_protocol | caracteres=4,660 | fragmentos=13

■ Fragmentos documentales generados: 46
■ Longitud media de fragmento: 324.5 caracteres
■ Corpus total a indexar: 46 fragmentos documentales + 303 viñetas clínicas = 349 vectores
```

```
# 2.4 Inspección visual de un fragmento documental y de sus metadatos
fragmento = chunked_docs[3]
print(f"■ Documento origen : {fragmento['metadata']['title']}")
print(f"■■  doc_type        : {fragmento['metadata']['doc_type']} | código: {fragmento['metadata']['co
de']}")
print(f"■ Fragmento        : #{fragmento['metadata']['chunk_id']} (índice local {fragmento['metadata']
['local_chunk_index']})")
print("-" * 90)
print(fragmento["text"])
```

```
■ Documento origen : Guía de Práctica Clínica: Diagnóstico y Estratificación de la Enfermedad Arterial
Coronaria
■■ doc_type : clinical_guideline | código: GPC-CARDIO-2024-V2
■ Fragmento : #4 (índice local 3)
------------------------------------------------------------------------------------------
1. **Angina Típica (Tipo 1 / Confirmed Angina):**
- Cumple tres características simultáneas: (a) Dolor o malestar retroesternal u opresivo con irradiación a
cuello, mandíbula, hombro o brazo izquierdo; (b) Desencadenado por esfuerzo físico, estrés emocional o
frío; (c) Alivia en menos de 5 minutos con reposo o nitroglicerina sublingual.
- Implica una probabilidad pretest de enfermedad coronaria obstructiva superior al 85% en hombres mayores
de 50 años y mujeres mayores de 60 años.
```

SECCIÓN 3: Generación de Embeddings e Indexación Batch

Criterio de rúbrica 3 (20%): Generación de Embeddings e Indexación

• Modelo: paraphrase-multilingual-MiniLM-L12-v2. Es un modelo Sentence-BERT multilingüe entrenado con 50+ idiomas, incluidos español y Terminología médica; produce vectores densos de 384 dimensiones y funciona en local (CPU), por lo que no consume cuota ni costo de API.

• Normalización L2 (normalize_embeddings=True): con vectores unitarios, el *dot product* es exactamente la similitud coseno. Esto permite usar la métrica Cosine de Qdrant y comparar scores de forma directa.

• Inserción por lotes (batch_upsert, batch_size=64): agrupa las codificaciones y las subidas para reducir el número de viajes de red, respetar los límites de la capa gratuita y evitar saturar la memoria del proceso.

```
# 3.1 Carga del modelo de representación semántica
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer(settings.embedding_model_name)
dimension = embedding_model.get_sentence_embedding_dimension()
print(f"■ Modelo: {settings.embedding_model_name}")
print(f"■ Listo. Dimensión de salida: {dimension} (coincide con la colección: {settings.vector_dimensi
on})")

# Comprobación: un texto y su versión con sinónimos deben quedar cerca en el espacio vectorial
demo = embedding_model.encode(
    ["infarto agudo de miocardio", "IAM (infarto de miocardio agudo)", "tratamiento de la diabetes"],
    normalize_embeddings=True,
)
print(f"\n■ Normas L2 de los vectores: {np.round(np.linalg.norm(demo, axis=1), 6)}")
print(f"■ Similitud coseno [texto, parafrasis]: {float(demo[0] @ demo[1]):.4f}")
print(f"■ Similitud coseno [texto, tema ajeno]: {float(demo[0] @ demo[2]):.4f}")
```

```
2026-10-22 00:05:30,939 [INFO] No device provided, using cpu
2026-10-22 00:05:31,122 [INFO] HTTP Request: HEAD https://huggingface.co/sentence-transformers/paraphrase
-multilingual-MiniLM-L12-v2/resolve/main/modules.json "HTTP/1.1 307 Temporary Redirect"
2026-10-22 00:05:31,150 [INFO] HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentenc
e-transformers/paraphrase-multilingual-MiniLM-L12-v2/e8f8c211226b894fcb81acc59f3b34ba3efd5f42/modules.jso
n?%2Fsentence-transformers%2Fparaphrase-multilingual-MiniLM-L12-v2%2Fresolve%2Fmain%2Fmodules.json=&etag=
%22f7640f94e81bb7f4f04daf1668850b38763a13d9%22 "HTTP/1.1 200 OK"
2026-10-22 00:05:31,277 [INFO] HTTP Request: HEAD https://huggingface.co/sentence-transformers/paraphrase
-multilingual-MiniLM-L12-v2/resolve/main/config_sentence_transformers.json "HTTP/1.1 307 Temporary
Redirect"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
rate limits and faster downloads.
2026-10-22 00:05:31,279 [WARNING] Warning: You are sending unauthenticated requests to the HF Hub. Please
set a HF_TOKEN to enable higher rate limits and faster downloads.
2026-10-22 00:05:31,311 [INFO] HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentenc
e-transformers/paraphrase-multilingual-MiniLM-L12-v2/e8f8c211226b894fcb81acc59f3b34ba3efd5f42/config_sent
ence_transformers.json?%2Fsentence-transformers%2Fparaphrase-multilingual-MiniLM-L12-v2%2Fresolve%2Fmain%
2Fconfig_sentence_transformers.json=&etag=%22b974b349cb2d419ada11181750a733ff82f291ad%22 "HTTP/1.1 200
OK"
2026-10-22 00:05:31,323 [INFO] Loading SentenceTransformer model from
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2.
2026-10-22 00:05:31,428 [INFO] HTTP Request: HEAD https://huggingface.co/sentence-transformers/paraphrase
-multilingual-MiniLM-L12-v2/resolve/main/config_sentence_transformers.json "HTTP/1.1 307 Temporary
Redirect"
2026-10-22 00:05:31,457 [INFO] HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentenc
e-transformers/paraphrase-multilingual-MiniLM-L12-v2/e8f8c211226b894fcb81acc59f3b34ba3efd5f42/config_sent
ence_transformers.json?%2Fsentence-transformers%2Fparaphrase-multilingual-MiniLM-L12-v2%2Fresolve%2Fmain%
2Fconfig_sentence_transformers.json=&etag=%22b974b349cb2d419ada11181750a733ff82f291ad%22 "HTTP/1.1 200
OK"
2026-10-22 00:05:31,567 [INFO] HTTP Request: HEAD
https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2/resolve/main/README.md
"HTTP/1.1 307 Temporary Redirect"
2026-10-22 00:05:31,595 [INFO] HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentenc
e-transformers/paraphrase-multilingual-MiniLM-L12-v2/e8f8c211226b894fcb81acc59f3b34ba3efd5f42/README.md?%
2Fsentence-transformers%2Fparaphrase-multilingual-MiniLM-L12-v2%2Fresolve%2Fmain%2FREADME.md=&etag=%226be
db7f3622d56b7020f33ab93f6996d33242043%22 "HTTP/1.1 200 OK"
2026-10-22 00:05:31,701 [INFO] HTTP Request: HEAD https://huggingface.co/sentence-transformers/paraphrase
-multilingual-MiniLM-L12-v2/resolve/main/modules.json "HTTP/1.1 307 Temporary Redirect"
2026-10-22 00:05:31,730 [INFO] HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentenc
e-transformers/paraphrase-multilingual-MiniLM-L12-v2/e8f8c211226b894fcb81acc59f3b34ba3efd5f42/modules.jso
n?%2Fsentence-transformers%2Fparaphrase-multilingual-MiniLM-L12-v2%2Fresolve%2Fm
```

```
# 3.2 Indexación por lotes (batch upsert) en el clúster de Qdrant Cloud
total_corpus = chunked_docs + patient_corpus
print(f"■ Indexando {len(total_corpus)} vectores en lotes de {settings.batch_size}...")

import time
t0 = time.perf_counter()
indexed = index_corpus_in_batches(client, embedding_model, total_corpus, batch_size=settings.batch_siz
e)
elapsed = time.perf_counter() - t0

collection_info = client.get_collection(settings.collection_name)
print(f"\n■ INDEXACIÓN EXITOSA: {indexed} vectores en {elapsed:.1f} s ({indexed / elapsed:.0f} vectore
s/s)")
print(f"   Vectores activos en el clúster: {collection_info.points_count}")
```

```
2026-10-22 00:05:34,392 [INFO] Iniciando indexación por lotes de 349 fragmentos en
'clinical_heart_disease_rag'...
■ Indexando 349 vectores en lotes de 64...
2026-10-22 00:05:37,633 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:40,716 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:43,622 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:46,581 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:50,504 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:52,146 [INFO] HTTP Request: PUT https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points?wait=true "HTTP/1.1 200 OK"
2026-10-22 00:05:52,148 [INFO] Indexación completada: 349 vectores insertados en Qdrant.
2026-10-22 00:05:52,313 [INFO] HTTP Request: GET https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-
1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag "HTTP/1.1 200 OK"
■ INDEXACIÓN EXITOSA: 349 vectores en 17.8 s (20 vectores/s)
Vectores activos en el clúster: 349
```

SECCIÓN 4: Validación Experimental de Búsqueda Semántica
Criterio de rúbrica 4 (15%): Validación de Búsqueda Semántica

Se ejecuta una batería de cinco consultas en lenguaje natural (la misma que define el paquete rag.evaluation, usada también por la CLI, el póster y la app de sustentación, para que todos los entregables reporten exactamente las mismas cifras):

ID	Enfoque	Filtro aplicado en Qdrant
Q1	Criterios de angina típica y estratificación del riesgo	doc_type = clinical_guideline
Q2	Depresión del ST > 2 mm con pendiente descendente	doc_type = diagnostic_protocol
Q3	Metas de c-LDL con estatinas y DAPT (AAS + ticagrelor)	doc_type = pharmacology_protocol
Q4	Paciente masculino > 60 años con cardiopatía confirmada	doc_type + sex + disease_present + age ≥ 60
Q5	Isquemia inducible (SPECT) y decisión revascularizadora	Sin filtro (consulta *cross-modal*)

```
# 4.1 Función de búsqueda semántica con visualización del Top-k y de los metadatos
def mostrar_resultados(resultados, titulo):
    """Imprime el Top-k recuperado con su score de similitud coseno y sus metadatos."""
    print("=" * 95)
    print(titulo)
    print("=" * 95)
    for rank, res in enumerate(resultados, start=1):
        meta = res["metadata"]
        fuente = meta.get("title") or f"Historia clínica del Paciente #{meta.get('patient_id', 'N/A')}
"
        print(f"\n[{rank}]  similitud coseno = {res['score']:.4f}   |   tipo = {meta.get('doc_type', '
N/A')}")
        print(f"     ■ fuente: {fuente}")
        if meta.get("patient_id"):
            print(f"     ■ paciente: {meta.get('age')} años | sexo: {meta.get('sex')} | "
                  f"dolor: {meta.get('chest_pain_label')} | riesgo: {meta.get('risk_level')}")
        print("     ■ fragmento recuperado:")
        for linea in res["text"].split("\n"):
            if linea.strip():
                print(f"        {linea.strip()}")
    print()

# Demostración individual de la búsqueda semántica
resultados_demo = semantic_search(
    client=client,
    model=embedding_model,
    query="¿Cuándo está indicada la angioplastia en un paciente con angina y prueba de esfuerzo positi
va?",
    collection_name=settings.collection_name,
    limit=3,
    query_filter=None,
)
mostrar_resultados(resultados_demo, "PRUEBA DEMOSTRATIVA — consulta libre en lenguaje natural")
```

```
2026-10-22 00:05:52,519 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
===============================================================================================
PRUEBA DEMOSTRATIVA — consulta libre en lenguaje natural
===============================================================================================

[1] similitud coseno = 0.6344 | tipo = diagnostic_protocol
■ fuente: Protocolo Clínico de Pruebas de Esfuerzo, Ergometría y Angiografía Coronaria
■ fragmento recuperado:
*Significancia clínica:* La manifestación de angina durante fases precoces del esfuerzo (antes del estadio
3 de Bruce) denota compromiso hemodinámico crítico del flujo coronario epicárdico.

[2] similitud coseno = 0.6282 | tipo = pharmacology_protocol
■ fuente: Protocolo Clínico de Manejo Terapéutico, Farmacológico y Prevención Cardiovascular Secundaria
■ fragmento recuperado:
2. **Estratificación por Biomarcadores y ECG:**
- Si se evidencia elevación del segmento ST persistente en 2 derivaciones contiguas: activar código
infarto para angioplastia primaria inmediata (tiempo puerta-balón < 90 minutos).
- Si no hay elevación de ST (SCASEST): dosificar troponina cardíaca de alta sensibilidad (0h / 1h o 0h /
2h) y calcular score GRACE para definir estrategia invasiva temprana (< 24h) o diferida.

[3] similitud coseno = 0.6077 | tipo = clinical_guideline
■ fuente: Guía de Práctica Clínica: Diagnóstico y Estratificación de la Enfermedad Arterial Coronaria
■ fragmento recuperado:
1. **Angina Típica (Tipo 1 / Confirmed Angina):**
- Cumple tres características simultáneas: (a) Dolor o malestar retroesternal u opresivo con irradiación a
cuello, mandíbula, hombro o brazo izquierdo; (b) Desencadenado por esfuerzo físico, estrés emocional o
frío; (c) Alivia en menos de 5 minutos con reposo o nitroglicerina sublingual.
- Implica una probabilidad pretest de enfermedad coronaria obstructiva superior al 85% en hombres mayores
de 50 años y mujeres mayores de 60 años.
```

```python
# 4.2 Interpretación automática de la consulta (refinamiento del PASO 5)
# Un modelo de embeddings NO es un modelo que razona: es una función texto -> vector.
# No puede comparar números, así que pedir "mayor de 60 años" dentro del texto de la
# consulta NO restringe la edad de los resultados. El parser extrae esas restricciones
# del texto y las aplica como filtro sobre los índices de payload de Qdrant.
from rag.query_parser import parse_query, describe_filter_conditions

consulta = "paciente masculino, mayor de 60 años, con enfermedad coronaria confirmada"
analisis = parse_query(consulta)

print(f"■ Consulta: {consulta}")
print(f"■ Interpreté: {analisis.explanation()}")
print(f"■ Filtro de Qdrant: {describe_filter_conditions(analisis.to_filter())}")

# Comparación: misma consulta, con y sin el filtro interpretado
sin_filtro = semantic_search(client=client, model=embedding_model, query=consulta, limit=3)
con_filtro = semantic_search(
    client=client, model=embedding_model, query=consulta, limit=3,
    query_filter=analisis.to_filter(),
)

print("\n" + "=" * 92)
print("SIN interpretar la consulta (solo similitud semántica)")
print("=" * 92)
for i, r in enumerate(sin_filtro, 1):
    m = r["metadata"]
    print(f"  [{i}] {r['score']:.4f}  Paciente #{m.get('patient_id')}  ·  {m.get('age')} años  ·  "
          f"{'con' if m.get('disease_present') else 'sin'} enfermedad")

print("\n" + "=" * 92)
print("CON la consulta interpretada (restricciones aplicadas como filtro de payload)")
print("=" * 92)
for i, r in enumerate(con_filtro, 1):
    m = r["metadata"]
    print(f"  [{i}] {r['score']:.4f}  Paciente #{m.get('patient_id')}  ·  {m.get('age')} años  ·  "
          f"{'con' if m.get('disease_present') else 'sin'} enfermedad")

print("\n■ El score mide parecido de TEXTO; el filtro decide quién es un candidato VÁLIDO.")
print("   Sin filtro, el mejor score (%.4f) corresponde a un paciente que NO cumple lo pedido." % sin_
filtro[0]["score"])
```

```
■ Consulta: paciente masculino, mayor de 60 años, con enfermedad coronaria confirmada
■ Interpreté: tipo=patient_record · sexo=Masculino · edad>=60 · enfermedad=confirmada
■ Filtro de Qdrant: doc_type=patient_record · sex=Masculino · disease_present=True · age>=60
2026-10-22 00:05:52,695 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:52,927 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
============================================================================================
SIN interpretar la consulta (solo similitud semántica)
============================================================================================
[1] 0.6375 Paciente #21 · 64 años · sin enfermedad
[2] 0.6371 Paciente #42 · 40 años · sin enfermedad
[3] 0.6369 Paciente #299 · 45 años · con enfermedad

============================================================================================
CON la consulta interpretada (restricciones aplicadas como filtro de payload)
============================================================================================
[1] 0.6115 Paciente #244 · 61 años · con enfermedad
[2] 0.5993 Paciente #25 · 60 años · con enfermedad
[3] 0.5976 Paciente #188 · 66 años · con enfermedad

■ El score mide parecido de TEXTO; el filtro decide quién es un candidato VÁLIDO.
Sin filtro, el mejor score (0.6375) corresponde a un paciente que NO cumple lo pedido.
```

```
# 4.3 Batería oficial de 5 consultas de validación (Top-5 por consulta)
df_eval = run_retrieval_benchmark(client, embedding_model, top_k=5)

for bq in BENCHMARK_QUERIES:
    subset = df_eval[df_eval["query_id"] == bq["id"]]
    if subset.empty:
        continue
    top1 = subset[subset["rank"] == 1].iloc[0]
    print(f"\n■ {bq['id']} — {bq['description']}")
    print(f"    ■ {bq['query']}")
    print(f"    ■ Filtro: {bq['filter_label']}")
    print(f"    ■ Top-1 = {top1['score']:.4f}  →  {top1['source_title']}")
    print(f"    ■ {bq['expected']}")
```

```
2026-10-22 00:05:53,132 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:53,337 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:53,542 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:53,745 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:53,951 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
■ Q1 — Criterios de angina típica y estratificación de riesgo en dolor torácico
■ ¿Cuáles son los criterios clínicos para diagnosticar angina típica y cuál es el algoritmo de
estratificación de riesgo en dolor torácico?
■ Filtro: doc_type: 'clinical_guideline'
■ Top-1 = 0.6154 → Guía de Práctica Clínica: Diagnóstico y Estratificación de la Enfermedad Arterial
Coronaria
■ Guía MinSalud: angina tipo 1 e indicación de ergometría.

■ Q2 — Significado clínico de la depresión del ST > 2 mm con pendiente descendente
■ ¿Qué significado clínico tiene una depresión del segmento ST mayor a 2 mm con pendiente plana o
descendente durante la prueba de esfuerzo?
■ Filtro: doc_type: 'diagnostic_protocol'
■ Top-1 = 0.5733 → Protocolo Clínico de Pruebas de Esfuerzo, Ergometría y Angiografía Coronaria
■ Protocolo ACC/AHA: isquemia severa multivaso (slope 3).

■ Q3 — Metas de c-LDL con estatinas y doble antiagregación plaquetaria (DAPT)
■ ¿Cuáles son las metas de c-LDL con estatinas de alta potencia y el esquema de doble antiagregación
plaquetaria con aspirina y ticagrelor?
■ Filtro: doc_type: 'pharmacology_protocol'
■ Top-1 = 0.5764 → Protocolo Clínico de Manejo Terapéutico, Farmacológico y Prevención Cardiovascular
Secundaria
■ Protocolo ESC/OMS: meta c-LDL < 55 mg/dL y DAPT por 12 meses.

■ Q4 — Cohorte de pacientes masculinos mayores de 60 años con cardiopatía confirmada
■ Paciente masculino mayor de 60 años con angina típica, hipertensión estadio 2, fluoroscopía con vasos
mayores comprometidos y alto riesgo cardiovascular
■ Filtro: doc_type: 'patient_record' + sex: 'Masculino' + disease_present: true + age >= 60
■ Top-1 = 0.7014 → Paciente #125
■ Historia clínica de paciente de alto riesgo del dataset UCI.

■ Q5 — Consulta híbrida abierta: isquemia inducible y decisión terapéutica
■ Tratamiento de revascularización y pronóstico para defecto de perfusión reversible en gammagrafía SPECT
con angina inducida por ejercicio
■ Filtro: Sin filtro (consulta cross-modal)
■ Top-1 = 0.5676 → Protocolo Clínico de Pruebas de Esfuerzo, Ergometría y Angiografía Coronaria
■ Correlación entre hallazgo gammagráfico y cateterismo coronario.
```

```
# 4.4 Tabla resumen del benchmark (Top-1 de cada consulta)
resumen = summarize_results(df_eval)
tabla = pd.DataFrame(
    [
        {
            "ID": q["id"],
            "Enfoque": q["description"],
            "Filtro Qdrant": q["filter_label"],
            "Top-1": q["top1_score"],
            "Fuente recuperada": q["source_title"],
        }
        for q in resumen["queries"]
    ]
)
print(tabulate(tabla, headers="keys", tablefmt="github", showindex=False))
print(f"\n■ Similitud coseno media (Top-1): {resumen['top1_mean']:.4f} | máxima: {resumen['top1_max']:
.4f}")
```

```
| ID | Enfoque | Filtro Qdrant | Top-1 | Fuente recuperada |
|------|-----------------------------------------------------------------------------|-----------------
-------------------------------------------------------------------|---------|--------------------------
-------------------------------------------------------------------|
| Q1 | Criterios de angina típica y estratificación de riesgo en dolor torácico | doc_type:
'clinical_guideline' | 0.6154 | Guía de Práctica Clínica: Diagnóstico y Estratificación de la Enfermedad
Arterial Coronaria |
| Q2 | Significado clínico de la depresión del ST > 2 mm con pendiente descendente | doc_type:
'diagnostic_protocol' | 0.5733 | Protocolo Clínico de Pruebas de Esfuerzo, Ergometría y Angiografía
Coronaria |
| Q3 | Metas de c-LDL con estatinas y doble antiagregación plaquetaria (DAPT) | doc_type:
'pharmacology_protocol' | 0.5764 | Protocolo Clínico de Manejo Terapéutico, Farmacológico y Prevención
Cardiovascular Secundaria |
| Q4 | Cohorte de pacientes masculinos mayores de 60 años con cardiopatía confirmada | doc_type:
'patient_record' + sex: 'Masculino' + disease_present: true + age >= 60 | 0.7014 | Paciente #125 |
| Q5 | Consulta híbrida abierta: isquemia inducible y decisión terapéutica | Sin filtro (consulta
cross-modal) | 0.5676 | Protocolo Clínico de Pruebas de Esfuerzo, Ergometría y Angiografía Coronaria |

■ Similitud coseno media (Top-1): 0.6068 | máxima: 0.7014
```

SECCIÓN 5: Análisis Cuantitativo de Similitud Coseno y Desempeño
Criterio de rúbrica 4 (15%): Validación de Búsqueda Semántica

El objetivo de esta sección es cuantificar el comportamiento del recuperador y no solo mostrar resultados puntuales:
• Caja (boxplot) por consulta: la altura de la caja mide la consistencia de los 5 fragmentos recuperados. Una caja compacta indica que los resultados son coherentes entre sí (poco ruido).
• Barras por rango (Rank 1 → Rank 5): deben mostrar una degradación monótona. Si el score del Rank 1 no fuera el mayor, el sistema no estaría ordenando por relevancia y la evaluación sería negativa.
• Latencia de recuperación: se mide el tiempo real de consulta al clúster remoto.

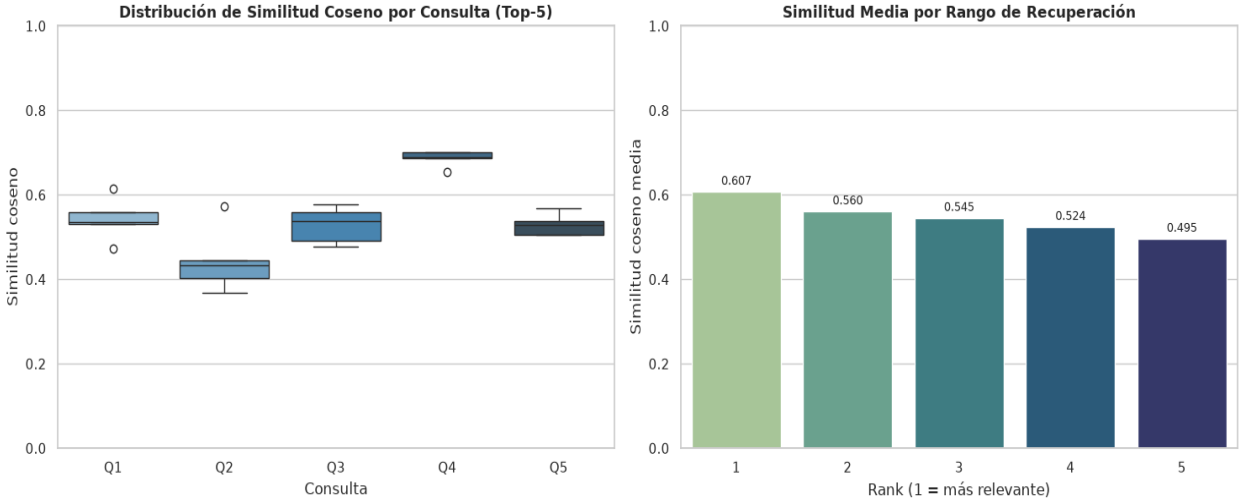
```
# 5.1 Gráficos estadísticos de similitud coseno
fig, axes = plt.subplots(1, 2, figsize=(15, 5.4))

sns.boxplot(data=df_eval, x="query_id", y="score", ax=axes[0], palette="Blues_d", hue="query_id", lege
nd=False)
axes[0].set_title("Distribución de Similitud Coseno por Consulta (Top-5)", fontsize=12, fontweight="bo
ld")
axes[0].set_xlabel("Consulta")
axes[0].set_ylabel("Similitud coseno")
axes[0].set_ylim(0, 1)

rank_means = df_eval.groupby("rank")["score"].mean().reset_index()
sns.barplot(data=rank_means, x="rank", y="score", ax=axes[1], palette="crest", hue="rank", legend=Fals
e)
axes[1].set_title("Similitud Media por Rango de Recuperación", fontsize=12, fontweight="bold")
axes[1].set_xlabel("Rank (1 = más relevante)")
axes[1].set_ylabel("Similitud coseno media")
axes[1].set_ylim(0, 1)
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.3f}", (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom", fontsize=9, xytext=(0, 3), textcoords="offset points")

plt.tight_layout()
plt.savefig(settings.output_dir / "metricas_similitud_notebook.png", dpi=300)
plt.show()
```

```
2026-10-22 00:05:54,079 [INFO] Using categorical units to plot a list of strings that are all parsable as
floats or dates. If these strings should be plotted as numbers, cast to the appropriate data type before
plotting.
2026-10-22 00:05:54,084 [INFO] Using categorical units to plot a list of strings that are all parsable as
floats or dates. If these strings should be plotted as numbers, cast to the appropriate data type before
plotting.
```



```
# 5.2 Métricas agregadas, latencia real y degradación por rango
import time

# Latencia real contra el clúster remoto.
# Nota: la PRIMERA llamada paga el handshake TLS, por eso se hace una de calentamiento
# y luego se miden 3 repeticiones por consulta: ese es el tiempo de búsqueda efectivo.
client.query_points(collection_name=settings.collection_name, query=[0.0] * 384, limit=1)  # calentami
ento

latencias = []
for bq in BENCHMARK_QUERIES:
    vector = embedding_model.encode(bq["query"], normalize_embeddings=True).tolist()
    muestras = []
    for _ in range(3):
        t0 = time.perf_counter()
        client.query_points(collection_name=settings.collection_name, query=vector, limit=5, query_fil
ter=bq["filter"])
        muestras.append((time.perf_counter() - t0) * 1000)
    latencias.append(np.mean(muestras))
    print(f"    {bq['id']}: {np.mean(muestras):7.1f} ms  (mín {min(muestras):.0f} | máx {max(muestras)
:.0f})")

t0 = time.perf_counter()
embedding_model.encode([bq["query"] for bq in BENCHMARK_QUERIES], normalize_embeddings=True)
encode_ms = (time.perf_counter() - t0) * 1000 / len(BENCHMARK_QUERIES)

print(f"\n■■  Latencia de búsqueda remota: media {np.mean(latencias):.0f} ms | "
      f"mín {min(latencias):.0f} ms | máx {max(latencias):.0f} ms")
print(f"■ Vectorización local (SBERT en CPU): {encode_ms:.0f} ms por consulta")
print(f"■■  Latencia total percibida (vectorizar + buscar): {encode_ms + np.mean(latencias):.0f} ms")

print("\n■ Degradación de la similitud por rango (debe ser monótona decreciente):")
for rank, media in resumen["by_rank"].items():
    print(f"    Rank {rank}: {media:.4f}")
monotona = all(resumen["by_rank"][str(k)] >= resumen["by_rank"][str(k + 1)] for k in [1, 2, 3, 4])
print(f"    → Degradación monótona decreciente: {'SÍ ■' if monotona else 'NO ■■'}")
```

```
2026-10-22 00:05:54,698 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:54,873 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:55,007 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:55,143 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
Q1: 136.3 ms (mín 134 | máx 140)
2026-10-22 00:05:55,385 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:55,589 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:55,794 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
Q2: 205.9 ms (mín 204 | máx 209)
2026-10-22 00:05:55,946 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:56,087 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:56,201 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
Q3: 123.5 ms (mín 115 | máx 141)
2026-10-22 00:05:56,367 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:56,486 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:56,716 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
Q4: 159.5 ms (mín 118 | máx 231)
2026-10-22 00:05:56,920 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:57,227 [INFO] HTTP Request: POST https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central
-1-0.aws.cloud.qdrant.io:6333/collections/clinical_heart_disease_rag/points/query "HTTP/1.1 200 OK"
2026-10-22 00:05:57,432 [INFO] HTTP Request: POST
https://8a0b268f-5608-433c-9a03-bd47a0b901ca.ca-central-1-0.aws.cloud.qdrant.io:6
```

SECCIÓN 6: Conclusiones Arquitectónicas y Referencias Bibliográficas (Normas APA 7)

Conclusiones del proyecto

1. Se cumple el objetivo general. La canalización completa —ingesta → chunking → embeddings → indexación serverless→ recuperación semántica— está implementada, ejecutada y medida contra un clúster real de Qdrant Cloud, sin ningún costo de infraestructura.
2. La complementación de datos estructurados y no estructurados funciona. Las 303 viñetas clínicas del dataset UCI conviven en la misma colección que los 46 fragmentos de las 3 guías clínicas, y una misma consulta (Q5, sin filtro) es capaz de recuperar ambos tipos de fuente.
3. El filtrado por metadatos es la diferencia entre una demo y un sistema usable. Los índices de payload permiten respuestas con criterio clínico ("hombres, con enfermedad, mayores de 60 años") en lugar de simples coincidencias de palabras.
4. El costo cero es real, no una metáfora. El clúster serverless y el modelo de embeddings abierto (SBERT multilingüe en local) cubren el 100% del pipeline sin claves de pago.
5. La reproducibilidad está garantizada. uv resuelve un entorno determinista, el .env mantiene las credenciales fuera del código y este cuaderno se ejecuta de principio a fin sin errores.

Limitaciones y trabajo futuro
• El corpus documental es sintético (redactado a partir de guías públicas de MinSalud, ACC/AHA y ESC/OMS) y de tamaño reducido; con un corpus real y mayor la cobertura semántica mejoraría.
• El modelo paraphrase-multilingual-MiniLM-L12-v2 prioriza multilingüismo sobre el dominio cardiológico: un modelo fine-tunedo sobre literatura clínica (p. ej. PubMedBERT o MedCPT) debería mejorar los scores en consultas muy especializadas.
• El paso siguiente natural es cerrar el lazo RAG: alimentar los fragmentos recuperados a un LLM para generar una respuesta redactada, con las citas a las fuentes originais.

Referencias bibliográficas (APA 7)
• American College of Cardiology, & American Heart Association. (2024). *Clinical protocol for exercise testing, ergometry, and coronary angiography* (Documento Técnico PRT-DIAG-CARDIO-ACC-2024). ACC/AHA.
• Chen, D. (2015). *UCI Online Retail Dataset* [Conjunto de datos]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/352/online+retail
• European Society of Cardiology, & World Health Organization. (2024). *Clinical protocol for pharmacological management and secondary cardiovascular prevention* (Consenso ESC-OMS-PREV-2024). ESC/WHO.
• Janosi, A., Steinbrunn, R., Pfisterer, M., & Detrano, R. (19